Assignment 3: Database Design Analysis

Functional dependencies, redundancy and anomalies

Project Topic: Hotel Management System

1. Introduction

The Hotel Management System database stores guest details, room information, staff records, reservations, payments and extra services. It uses eight tables: Guests, RoomTypes, Rooms, Staff, Reservations, Payments, Services and ServiceOrders. This analysis examines redundancy, functional dependencies, anomalies and improvements.

2. Data Redundancy

Data redundancy means storing the same information more than once. If guest names and phone numbers were copied into every reservation, the same facts would be repeated. If room type names and standard prices were repeated in the Rooms table, a price change could require many updates. The separate Guests and RoomTypes tables reduce these repetitions. Similarly, Services stores each service name and standard price once, while ServiceOrders stores each service request.

3. Functional Dependencies

A functional dependency means that one attribute determines another attribute. In the following list, each primary key determines the other non-key attributes in its table. The unique room_number also identifies a room.

Table	Functional dependency
Guests	guest_id → first_name, last_name, email, phone, address, id_proof_type, id_proof_number
RoomTypes	room_type_id → type_name, base_price, max_occupancy, description
Rooms	room_id → room_number, floor_number, room_type_id, status
Staff	staff_id → first_name, last_name, designation, phone, shift
Reservations	reservation_id → guest_id, room_id, staff_id, check_in_date, check_out_date, num_guests, booking_status
Payments	payment_id → reservation_id, amount, payment_date, payment_mode, payment_status
Services	service_id → service_name, price
ServiceOrders	service_order_id → reservation_id, service_id, quantity, order_date

Foreign keys connect related tables. For example, Reservations.guest_id refers to Guests.guest_id, and Rooms.room_type_id refers to RoomTypes.room_type_id.

4. Insertion, Deletion and Update Anomalies

4.1 Insertion anomaly

An insertion anomaly happens when new information cannot be added without unrelated information. If room type information existed only in Reservations, the hotel might be unable to add a new room type until a reservation was made. Keeping room types in their own table avoids this problem.

4.2 Deletion anomaly

A deletion anomaly happens when deleting one record also removes useful information. If service details were stored only in ServiceOrders, deleting the last order for a service could remove the only record of that service. The separate Services table protects the service catalogue.

4.3 Update anomaly

An update anomaly happens when repeated copies of the same information are not all changed. If a room type's price were repeated in many rows, changing only some rows would produce inconsistent prices. Storing the standard price in RoomTypes avoids that repetition.

5. How Poor Design Causes Anomalies

Combining different subjects in one table or repeating facts across many rows makes a database difficult to maintain. It can create inconsistent contact details and prices, prevent independent insertion of new information, or cause important information to be lost when a record is deleted. Primary keys, foreign keys and normalization reduce these risks.

6. Recommended Improvements

1. Keep the eight tables separated by subject. 2. Use primary keys and foreign keys to identify and connect records. 3. Apply NOT NULL, UNIQUE and CHECK constraints to validate important values. 4. Check that check-out is later than check-in and that guest counts and quantities are positive. 5. Prevent overlapping active reservations for the same room using suitable application logic or database transactions. 6. Store the rate agreed at booking in Reservations so later changes to the standard room price do not rewrite historical bookings. 7. Store each payment transaction separately.

7. Conclusion

The current design reduces unnecessary repetition by separating guest, room, staff, reservation, payment and service information. Its functional dependencies show how primary keys determine other attributes. Adding validation and booking rules will further improve consistency and reduce the chance of anomalies.